

Documentação de Projeto: FitHub Treino

Ecossistema: FitHub - Ecossistema de Aplicativos Fitness

Desenvolvedor: Rafael

Área de Atuação: Planejamento, execução e histórico de treinos

1. Visão Geral do Aplicativo (A Ideia)

O FitHub Treino será um aplicativo fitness independente, desenvolvido em Flutter, focado em ajudar o usuário a montar, acompanhar e registrar seus treinos de musculação ou exercícios físicos. A proposta é funcionar como um guia simples de treino, permitindo que a pessoa consulte exercícios, organize sua rotina semanal e acompanhe sua evolução.

Como vai funcionar na prática:

- **Biblioteca de exercícios:** o app exibirá uma lista de exercícios separados por grupo muscular, como peito, costas, pernas, ombro, bíceps, tríceps e abdômen. Cada exercício poderá mostrar nome, descrição, músculo trabalhado, equipamento necessário e dificuldade.
- **Montagem de treino:** o usuário poderá criar fichas de treino, por exemplo "Treino A - Peito e Tríceps", escolhendo exercícios, número de séries, repetições e carga sugerida.
- **Registro de execução:** durante o treino, o usuário marca os exercícios concluídos e informa a carga usada. O app salva o histórico para comparar a evolução ao longo dos dias.
- **Cronômetro de descanso:** após finalizar uma série, o app poderá iniciar um temporizador de descanso, por exemplo 60 ou 90 segundos, ajudando o usuário a manter o ritmo do treino.
- **Diferencial inteligente:** o app poderá sugerir exercícios com base no músculo escolhido e no equipamento disponível, usando dados vindos de uma API gratuita.

2. API Gratuita e Integração com Node.js

O aplicativo consumirá uma API Node.js própria. Essa API será responsável por organizar as informações e entregar respostas em JSON para o Flutter. Para obter a base de exercícios, a API Node.js poderá consultar uma API externa gratuita e aberta.

- **API externa sugerida:** wger API, uma API aberta de exercícios, músculos e equipamentos. Ela é adequada para projetos acadêmicos porque oferece dados estruturados e pode retornar respostas em JSON.
- **Função da API Node.js:** servir como intermediária entre o aplicativo Flutter e a API externa. Assim, o app não fica dependente diretamente da API de terceiros e o backend pode filtrar, traduzir ou adaptar os dados antes de enviar ao usuário.
- **Exemplo de resposta JSON:** nome do exercício, categoria, grupo muscular, equipamento, descrição e nível de dificuldade.
- **Endpoints possíveis da API Node.js:** **GET /exercicios**, **GET /exercicios?musculo=peito**, **POST /treinos**, **POST /historico**.

3. Stack Tecnológica

Camada	Tecnologia	Uso no projeto
Mobile / Frontend	Flutter (Dart)	Criação do aplicativo híbrido para Android e iOS com uma única base de código.
Backend	Node.js + Express	API própria para fornecer dados em JSON, salvar treinos e consultar exercícios.
API externa	wger API	Consulta de exercícios, músculos, categorias e equipamentos.

Banco local	Hive ou sqflite	Salvar treinos e histórico no celular, permitindo uso offline.
Banco em nuvem	Supabase PostgreSQL	Guardar perfil, fichas de treino e histórico em nuvem.

4. Ambiente de Desenvolvimento (IDE)

- Antigravity: o desenvolvimento poderá ser conduzido no ambiente Antigravity, utilizando IA agêntica para acelerar a criação das telas Flutter, a estruturação do backend Node.js e a documentação do projeto.
- VS Code ou Android Studio: também podem ser usados para execução, testes, emulador Android e depuração do app.

5. Arquitetura do Projeto

- Padrão Arquitetural: MVVM (Model-View-ViewModel). A View representa as telas do Flutter, o ViewModel controla os estados e regras de exibição, e os Models representam exercícios, treinos e registros de histórico.
- Services/Repositories: camada responsável por acessar a API Node.js, salvar dados no banco local e sincronizar com a nuvem.
- Gerência de Estado: Provider ou Riverpod, permitindo atualizar a tela quando o usuário adiciona exercício, conclui uma série ou altera a ficha de treino.
- Organização sugerida: pastas como **models**, **views**, **viewmodels**, **services**, **repositories** e **database**.

6. Recursos Nativos do Aparelho

O aplicativo utilizará recursos nativos para cumprir a exigência do projeto e melhorar a experiência do usuário:

- Notificações locais: envio de lembretes para o usuário treinar nos dias configurados, por exemplo "Hoje é dia de Treino A".
- Temporizador / cronômetro: controle do tempo de descanso entre séries, com alerta sonoro ou vibração ao final.
- Vibração: aviso discreto quando o tempo de descanso acabar.
- Armazenamento local: uso do banco de dados interno do aparelho para manter fichas e histórico mesmo sem internet.

7. Banco de Dados e Infraestrutura

- Banco de Dados Local (Offline-First): o app usará Hive ou sqflite para armazenar fichas de treino, exercícios favoritos, séries, repetições, cargas e treinos concluídos. Isso garante que o usuário consiga treinar mesmo sem conexão.
- Banco de Dados em Nuvem: o Supabase, baseado em PostgreSQL, poderá guardar os dados principais do usuário, permitindo recuperar as informações caso ele troque de celular.
- Sincronização: quando houver internet, o aplicativo envia os registros locais para a API Node.js, que salva no Supabase. O fluxo pode ser simples: Flutter API Node.js Supabase.
- Tabelas sugeridas: usuarios, exercicios_favoritos, treinos, treino_exercicios, historico_treinos e historico_series.

8. Funcionalidades Principais

- Tela inicial com resumo do treino do dia e botão para iniciar treino.
- Tela de biblioteca de exercícios com filtro por músculo e equipamento.
- Tela para criar e editar fichas de treino.
- Tela de execução do treino, marcando séries concluídas e carga utilizada.

-
- Histórico com evolução de cargas e treinos realizados.
 - Favoritos para salvar exercícios usados com frequência.

9. Exemplo de Fluxo do Usuário

1. O usuário abre o app e escolhe criar uma nova ficha de treino.
2. Ele seleciona o grupo muscular, por exemplo peito e tríceps.
3. O app busca exercícios na API Node.js, que consulta ou utiliza dados da wger API.
4. O usuário escolhe os exercícios e define séries, repetições e descanso.
5. No dia do treino, ele inicia a ficha, marca as séries concluídas e registra a carga usada.
6. Ao terminar, o app salva o histórico localmente e sincroniza com o banco em nuvem quando houver internet.

10. Conclusão

O FitHub Treino será um aplicativo simples, útil e viável para o projeto, pois combina Flutter, recurso nativo do aparelho, API Node.js com JSON, banco de dados local e banco em nuvem. A proposta atende ao objetivo do ecossistema FitHub, mantendo cada app independente, mas conectado ao mesmo tema fitness.

Fontes de apoio: documentação e repositório do projeto wger; documentação Supabase; documentação Flutter.